Greedy Algorithm: Kruskal's Algorithm

Design and Analysis of Algorithms

Dhannya SM

Dept of CSE
SSN College of Engineering

Problem Statement

- This is another algorithm to solve the MST problem.
- The algorithm constructs a minimum spanning tree as an expanding sequence of subgraphs that are always acyclic but are not necessarily connected on the intermediate stages of the algorithm.
- The algorithm begins by sorting the graph's edges in non-decreasing order of their weights.
- Then, starting with the empty subgraph, it scans this sorted list, adding the next edge on the list to the current subgraph if such an inclusion does not create a cycle.
- If it creates a cycle, it simply skips the edge.

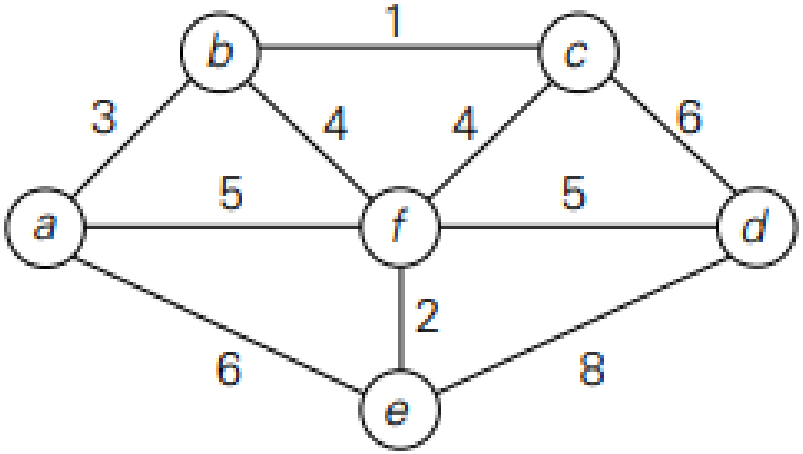
Example

//Kruskal's algorithm for constructing a minimum spanning tree
//Input: A weighted connected graph $G = \langle V, E \rangle$
//Output: E_T, the set of edges composing a minimum spanning tree of G
sort E in nondecreasing order of the edge weights $w(e_{i_1}) \leq \cdots \leq w(e_{i_{|E|}})$

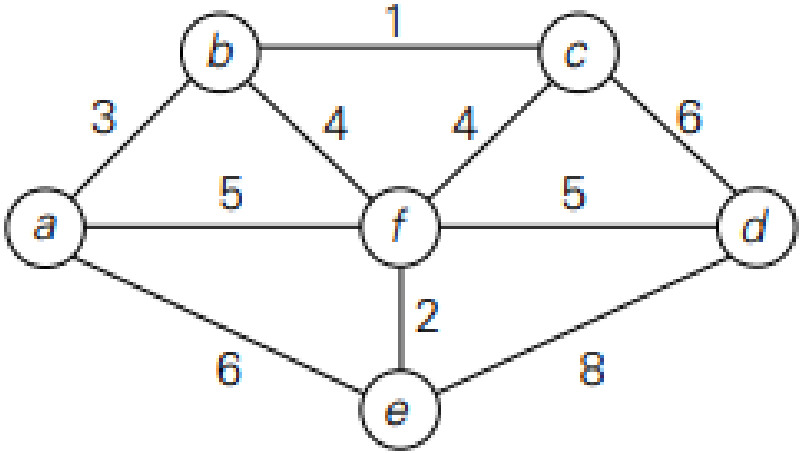
ALGORITHM *Kruskal(G)*

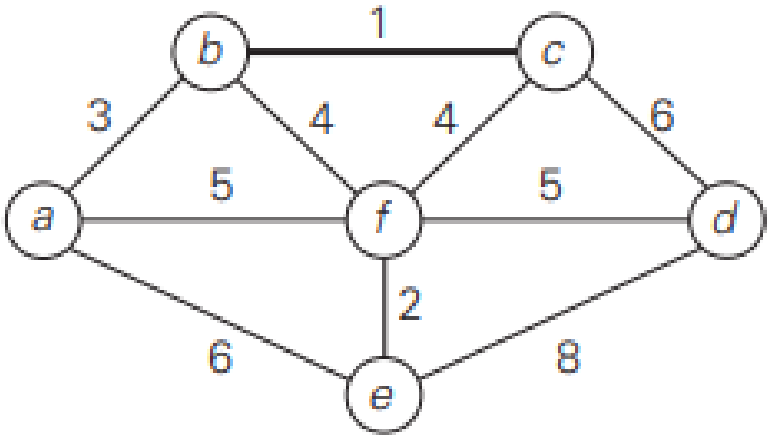
$E_T \leftarrow \varnothing$; $ecounter \leftarrow 0$ //initialize the set of tree edges and its size
$k \leftarrow 0$ //initialize the number of processed edges
while $ecounter < |V| - 1$ **do**
  $k \leftarrow k + 1$
  if $E_T \cup \{e_{i_k}\}$ is acyclic
    $E_T \leftarrow E_T \cup \{e_{i_k}\}$; $ecounter \leftarrow ecounter + 1$
return E_T

Algorithm

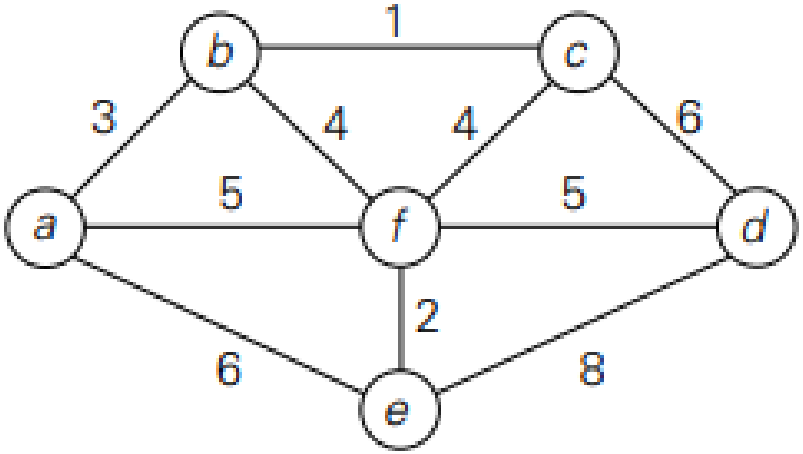


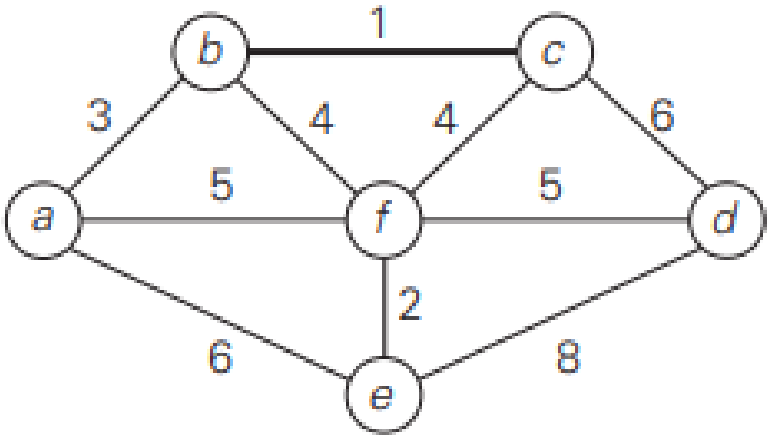
Algorithm

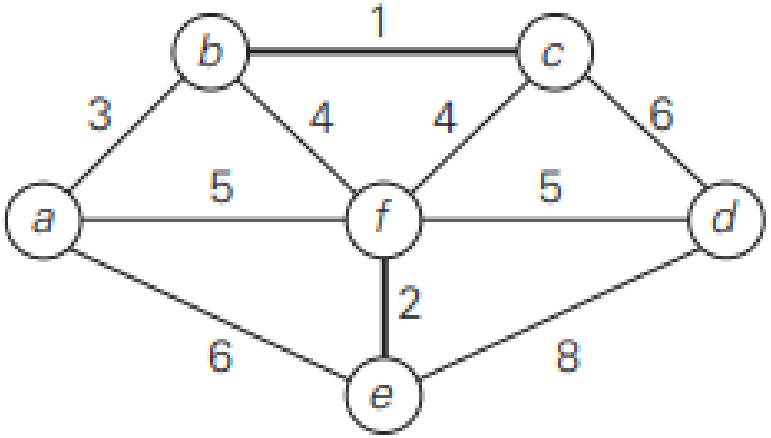




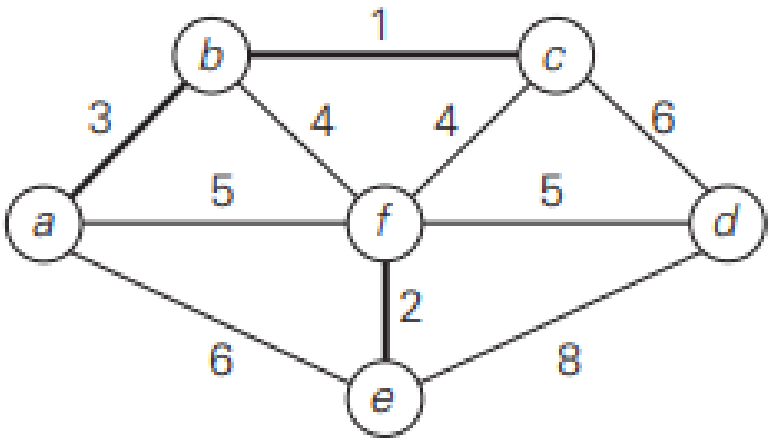
Algorithm



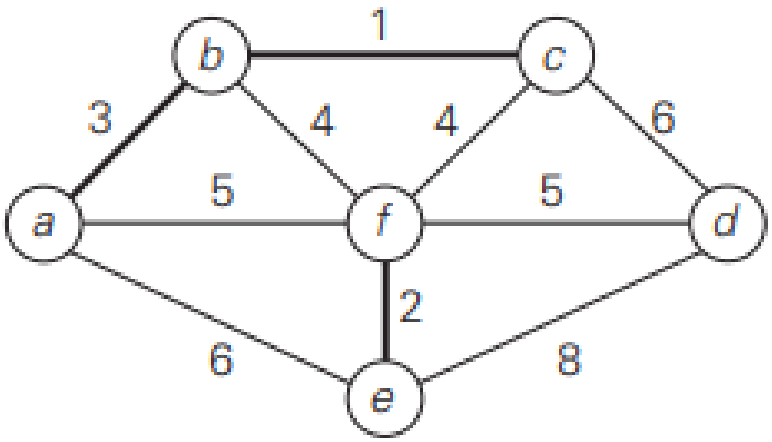


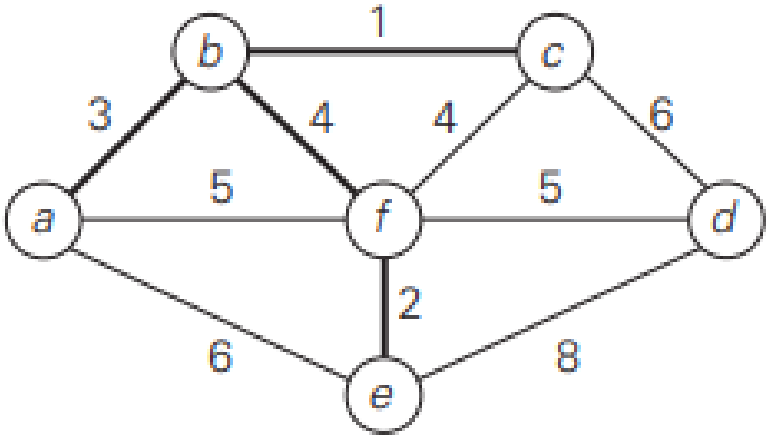


Algorithm execution

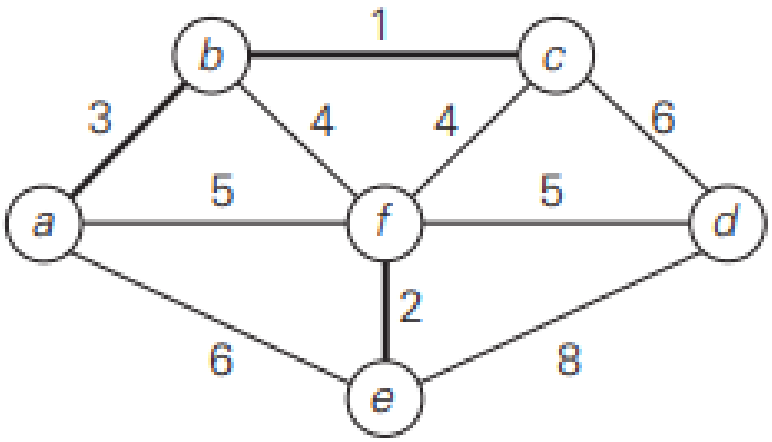


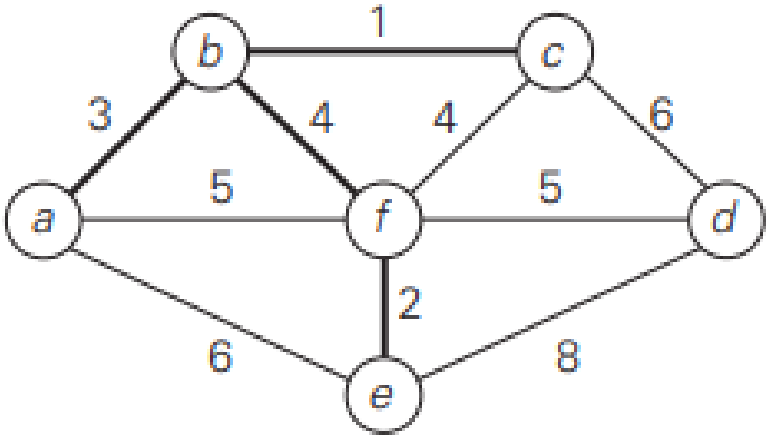
Algorithm execution

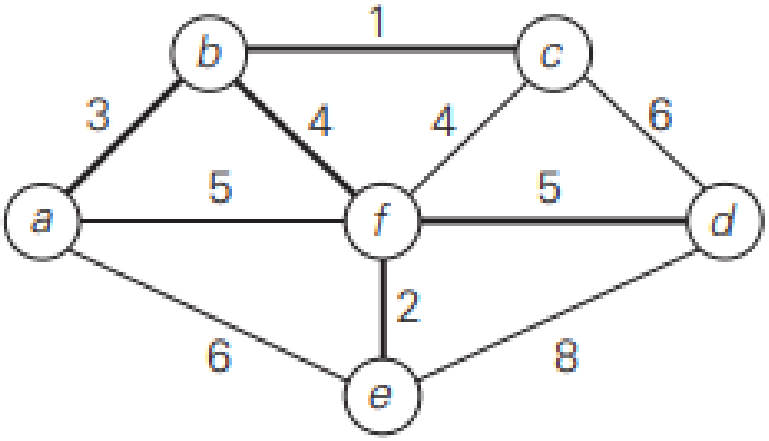




Algorithm execution







Union-Find Data Structure

- We can consider the algorithm's operations as a progression through a series of forests containing all the vertices of a given graph and some of its edges.
- The initial forest consists of $|V|$ trivial trees, each comprising a single vertex of the graph.
- The final forest consists of a single tree, which is a minimum spanning tree of the graph.
- On each iteration, the algorithm takes the next edge (u, v) from the sorted list of the graph's edges, finds the trees containing the vertices u and v, and, if these trees are not the same, unites them in a larger tree by adding the edge (u, v).

Union-Find Data Structure

- There are efficient algorithms for doing so, including the crucial check for whether two vertices belong to the same tree. They are called union-find algorithms.
- With an efficient union-find algorithm, the running time of Kruskal's algorithm will be dominated by the time needed for sorting the edge weights of a given graph. Hence, with an efficient sorting algorithm, the time efficiency of Kruskal's algorithm will be in $O(|E| \log |E|)$.

Union-Find Data Structure [1]

- A set S is partitioned into components $\{C_1, C_2, \ldots, C_k\}$.
- Each $s \in S$ belongs to exactly one C_j.
- It supports the following operations:
  - $Initialize(S)$ - set up initial singleton components $\{s\}$, for each $s \in S$
  - $Find(s)$ — return the component containing s
  - $Union(s, s')$ — merges components containing s, s'

[1] https://pdsaiitm.github.io/

Python implementation of UF Data Structure [2]

```python
class MakeUnionFind:
    def __init__(self):
        self.components = {}
        self.size = 0
    def initialize(self,vertices):
        self.size = vertices
        for vertex in range(vertices):
            self.components[vertex] = vertex
    def find(self,vertex):
        return self.components[vertex]
    def union(self,u,v):
        c_old = self.components[u]
        c_new = self.components[v]
        for k in range(self.size):
            if self.components[k] == c_old:
                self.components[k] = c_new
```

[2] https://pdsaiitm.github.io/

Time complexity of Union-Find

- Construct a minimum heap with m edges. This takes $O(|E|)$ or $O(|E| \log |E|)$ time.
- Extract one edge after the other and add it to the MST. Every time an edge is extracted, it is removed from the heap, and then 'heapfiy'. This takes $O(|E| \log |E|)$ for $|E|$ edges.
  - Check if both ends of an edge belong to the same component. If not, invoke union method.
  - Checking if a vertex belongs to a component takes $O(\log |E|)$.
  - Union is a constant time operation.

Summary

- Kruskal's algorithm is a greedy algorithm to find the minimum spanning tree of a graph.
- It uses the UNION-FIND data structure to compute the MST.
- The time complexity of Kruskal's algorithm is $O(|E| \log |E|)$.